



NATIVE COMPATIBILITY RECORD

Super Glove Ball Native Compatibility

Confirmed behavior, open questions, tracing, and safe fallback operation.

2026 EDITION / IAIN BENNETT / MIT LICENSE

This document records compatibility evidence for the VirtualGlove system's [lr-nestopia-powerglove](#) native core. It intentionally separates observations from hypotheses. Native detection, Start, continuous X/Y, orientation, stale-state safety, and live cabinet control are confirmed for the exact tested ROM. Signed Z and open/fist/index packet bytes are confirmed in exact-ROM headless traces. A completed live game also confirmed open-hand release/throw, fist grab/catch, index-point Robo-Bullet fire, and fist-plus-forward Power Punch. Movement is playable but retains some latency to refine. FCEUmm remains the supported explicit fallback using VirtualGlove's shared responsive D-pad and gesture recognition.

Camera intake before native emulation

Super Glove Ball does not receive frames directly. OpenCV or Direct V4L2 first delivers the newest camera picture to the same MediaPipe Hands graph used by every game. OpenCV is the production default because it provided the better live result on the tested Kiyo Pro. Direct V4L2 remains a capability-checked comparison for compatible 640×480 MJPEG cameras; it can provide driver timing and manual exposure controls but may be slower and falls back to OpenCV when unsupported.

One capture buffer favors minimum queue depth. Two can favor continuity with some cameras or hubs. Neither setting creates a movement history: one shared latest-frame slot replaces an unprocessed older frame, and the native receiver and core also consume latest state only. Camera choice, exposure, and buffers can therefore change frame age or tracking continuity without changing the ten-byte packet format, gesture mappings, calibration, or reach. Setup's camera test is the supported way to compare these choices on another camera.

Evidence order

When sources disagree, use this order:

1. The exact user-supplied Super Glove Ball ROM's input routines and control flow.
2. Controlled emulator traces of its writes, reads, assembled bytes, and cadence.
3. Repeatable in-game detection, out-of-range, and movement behavior.
4. Nestopia's existing Power Glove implementation.
5. The game manual and NESdev reverse-engineering notes.

NESdev material is a source of testable hypotheses, not a specification for this implementation.

Current compatibility status

Detail	Status	Evidence or next test
Shared camera recognition can supply continuous normalized X/Y	Confirmed in application tests	The authenticated receiver publishes the same calibrated axes used by gameplay.
A custom core can consume one coherent latest sample per emulated frame	Confirmed in build and unit tests	Versioned 64-byte read-only record with matching even guards; there is no queue or second smoothing stage.
Missing, uncalibrated, wrong-profile, or older-than-250 ms samples are neutral	Confirmed in implementation tests	The receiver also publishes a neutral record on transport timeout and shutdown.
The native core builds separately from stock Nestopia	Confirmed at pinned revision 5a1cd378cb46ca9ccc2dd6f8b2b6a79ab986052e	Linux builds identify as Nestopia PowerGlove ; the Windows build identifies as Nestopia VirtualGlove . Stock source and installed cores are not modified.
Candidate X/Y encoding reaches Nestopia's existing Power Glove device	Confirmed for the exact ROM	Minimum, center, and maximum X/Y each produced distinct packets. Cabinet validation corrected the camera-to-Nestopia Y orientation.
Detection signature, packet length, boundaries, and bit order	Confirmed	The ROM assembled inverse \$A0 as \$5F , strobed once per byte, read ten bytes/80 bits per sample MSB first, and required the final stored byte to be \$3F .
Start encoding	Confirmed	Native byte 6 value \$82 left the title screen and began play while the controller stayed in native mode. On Windows, exact-ROM traces confirm both the V-sign state and physical Player 1 Start produce that same native code.
Native Z encoding	Confirmed headlessly and in live gameplay	Calibrated camera depth is sign-reversed into the hardware convention. Neutral produced \$00 ; maximum forward motion produced \$81 . Fist plus forward motion triggered Power Punch during a completed game.
Native open, fist, and index-point encoding	Confirmed headlessly and in live gameplay	The exact ROM repeatedly received \$00 open, \$FF fist, and \$0F index-point samples. Shared five-finger recognition determines compound poses before transmission. Live play confirmed release/throw, grab/catch, and Robo-Bullet behavior.
Native roll byte and unobserved button codes	Neutral; no confirmed game action is missing	Native X/Y, depth, open hand, fist, index point, and Start are mapped. Super Glove Ball has shown no repeatable action for packet byte 4 or for other byte-6 codes. Standard A, B, Select, and wrist-to-button mappings remain available in the FCEUmm joystick mode; sending guessed native codes could create unintended input.
Poll timing tolerances	Confirmed for tested sessions	Headless runs sustained ten-byte polling throughout native phases, and live cabinet sessions remained stable. Broader hardware and timing stress coverage remains useful.
Headless X/Y activation and release responsiveness	Confirmed for the exact ROM	All four axes visibly diverged by frame 3; a 3.1% positive-X step also diverged by frame 3. See the direction-response benchmark .

Detail	Status	Evidence or next test
Cabinet field mapping and stabilization	Implemented and live-tested; synchronized physical latency remains open	Continuous X/Y uses fresh, geometry-validated MediaPipe palm observations, capture timestamps, per-player asymmetric reach, and an edge clamp. Latest coordinate is direct during continuous tracking and is the only live response path. Brief loss holds only X/Y for up to 180 ms and releases actions immediately. Strongly aligned forward recovery is accepted at once, while one contradictory or unusually distant non-forward recovery waits for the next fresh result. The bounded and optical-flow experiments are archived. FCEUmm D-pad thresholds are unchanged.
Portable defaults versus neutral calibration	Confirmed in application and installer tests	Full-field mapping, stabilization, and recognition thresholds ship in the release-owned profile baseline. The camera/player-specific neutral reference uses 24 geometrically valid observations and remains private across updates; handedness certainty is not treated as position confidence.
Explicit FCEUmm joystick fallback for the same ROM	Confirmed	The ROM enters play using standard Start, requests only the libretro joypad callback, and activates/releases every D-pad direction visibly by frame 3.

The validated ROM is [Super Glove Ball \(USA\)](#) with SHA-256 [ad60ef1b62cd1b3bc02a9320376067347a8ab2ebbe46e1616693d8379c9d9a7b](#). It remains outside the source tree and release packages. These results apply to that exact image; never silently turn a NESdev assumption into a compatibility claim for another revision.

Platform validation status

Platform	Native Super Glove Ball evidence
RetroPie	Completed cabinet play confirms detection, Start, continuous movement, depth, grab/throw, Robo-Bullet, and Power Punch.
Recalbox 10.1 rpizero2 on Raspberry Pi 3	Native gameplay, simultaneous physical-joypad use, and reboot persistence passed. Other packaged Recalbox targets retain their own hardware-validation requirement.
Batocera 43u.1 x86-64	The installer resolved, verified, and load-tested the packaged x86-64 core and pairing passed. Physical native gameplay on this machine has not yet been recorded.
LaunchBox on Windows x86-64	Native gameplay passed through the separately named DLL. VirtualGlove movement worked, and the physical gamepad supplied Start during diagnosis before the corrected V-sign Start path also passed.

These platform results supplement the exact-ROM software evidence above. They do not make one target's binary, controller mapping, or operating-system integration evidence for another target.

Confirmed exact-ROM packet

The ROM does not consume the full twelve-byte shape described by some secondary notes. It reads this ten-byte sample, with each transmitted bit inverted by the ROM while assembling the byte:

Byte	Confirmed use	Neutral/test values
0	Detection signature	\$A0 , assembled by the ROM as \$5F

Byte	Confirmed use	Neutral/test values
1	X	<code>\$80</code> minimum, <code>\$00</code> center, <code>\$7F</code> maximum
2	Y	<code>\$80</code> minimum, <code>\$00</code> center, <code>\$7F</code> maximum
3	Signed Z/depth	<code>\$00</code> neutral; forward camera motion maps toward <code>\$81</code> ; away maps positive
4	Unused roll candidate	<code>\$00</code> ; the exact ROM has shown no separate wrist-roll action
5	Hand gesture	<code>\$00</code> open, <code>\$FF</code> fist, <code>\$0F</code> index point
6	Button	<code>\$FF</code> neutral; <code>\$82</code> Start confirmed
7-8	No gameplay role established	Both remain <code>\$00</code> , matching Nestopia's fixed initialization
9	Validation terminator	<code>\$3F</code>

Nestopia initializes bytes 7-8 to `$00` and never updates them from controller input; our patch retains this behavior. Traces and completed live play confirm working input at these values, not that the ROM ignores them. No confirmed game action requires different values. Establishing a purpose would need focused ROM-use analysis or a repeatable one-byte-at-a-time gameplay test; these are not known missing controls.

The trace runner starts the exact ROM with the Power Glove attached, proves that native `$82` Start enters play, and holds each X/Y extreme for 120 frames. The captured screens place the Robo-Glove at left, center, right, bottom, center, and top respectively. Separate 60-frame phases then transmit open, fist, open, index-point, open, and fist-plus-forward-Z packets. Tracking-lost, uncalibrated, and stale phases prove that the core returns neutral axes, pose, and buttons instead of retaining the last sample.

Latest-sample interface

The authenticated console receiver owns `/run/virtualglove/native-state` on Linux; LaunchBox uses its per-user mapped record. It creates the record read-only for consumers. Format version 1 is a fixed 64-byte little-endian record containing:

- magic, format version, record size, and matching begin/end coherence guards;
- sample sequence and a console monotonic timestamp taken at publication; for Super Glove Ball the native record is written immediately after receiver validation. LaunchBox does not duplicate this native state through its ordinary-game Network RetroPad path;
- signed normalized X, Y, Z, and roll axes;
- detected and calibrated flags;
- four compact finger-flex levels;
- recognized-button and compound-pose mask, including five-finger fist and index-point decisions made by the shared recognizer;
- active-profile identifier;
- reserved bytes that stay zero.

The writer publishes an odd in-progress guard and then an even complete guard. The core copies one record at the beginning of its input callback and rejects it unless both guards match and are even. It consumes only current, calibrated, detected Super Glove Ball samples. Stale or invalid input leaves the emulated device neutral.

That timestamp is not the socket receive time or the core-consumption time. The [live baseline procedure](#) keeps those stages separate before movement-latency tuning.

Build and install the native core

The native core is a supported VirtualGlove system component. Its build script clones the official libretro Nestopia repository at the pinned revision into a dedicated build directory, applies the local patch, and emits [nestopia_powerglove_libretro.so](#) without changing a stock installation:

```
SH
scripts/build-nestopia-powerglove.sh
```

The normal RetroPie installer offers this source build when it finds a registered Super Glove Ball ROM. Accepting installs Git and standard build tools, builds in a temporary directory, installs the core under its separate name, copies the upstream GPLv2 [COPYING](#) file beside it, and adds the native entry to the launch menu. It deliberately leaves that ROM's current FCEUmm selection unchanged.

Batocera cores are target-specific and its `/usr` tree is read-only. Release packages carry Batocera 43.1 builds and complete corresponding source for all 15 supported targets. Startup resolves the exact architecture, verifies the manifest and ELF identity, and loads the selected shared object on the console before reversible overlays add the separate core and info entry. An exact release build is preferred; a newer Batocera release may try the newest packaged build for that architecture only because the target-side load check is the final gate. Failure leaves FCEUmm available. Exact registered Super Glove Ball ROMs are selected automatically only when no explicit Batocera core choice exists. Maintainers reproduce or resume the matrix with [scripts/build-batocera-native-matrix.sh](#) `/path/to/batocera.linux`; the manual single-target installer remains available for reviewed development builds.

Recalbox likewise requires an exact target build. Recalbox 10.1 packages include separate cores and complete source archives for [rpizero2](#), [rpi3](#), [rpi4_64](#), [rpi5_64](#), [rg353x](#), [odroidgo2](#), and [x86_64](#); the official image on the tested Raspberry Pi 3 reports [rpizero2](#). Maintainers can reproduce one target with [scripts/build-recalbox-nestopia-powerglove.sh](#) `/path/to/recalbox TARGET`. The installer prefers an exact Recalbox release build, otherwise selects the newest packaged build from the same major series. It verifies the packaged checksum, exact target, same-major compatibility, ELF identity, libretro API, and core name on Recalbox before keeping it in the persistent share and exposing it through a reversible runtime core overlay. A temporary system list adds the separate core to Recalbox's NES choices; the ROM-specific [.recalbox.conf](#) selects it without changing stock Nestopia or any other game. Normal startup finds exact Super Glove Ball filenames in the installed game registry and creates this selection only when a matching ROM has no existing [nes.core](#) choice. Existing choices are preserved, and a registry/discovery problem does not prevent VirtualGlove's receiver or game monitor from starting.

LaunchBox uses a Windows x86-64 DLL built from the same pinned Nestopia source. The common native patch is followed by [native/launchbox/nestopia-windows.patch](#), which uses Windows file mapping and the high-resolution performance counter for the guarded sample record. The LaunchBox wrapper selects [nestopia_powerglove_libretro.dll](#) only for exact registered Super Glove Ball filenames; all other NES games retain FCEUmm. The DLL is separately named and does not replace RetroArch's stock Nestopia core. The Windows installer verifies the DLL and corresponding source, actually loads it, and confirms the libretro identity before installation. The wrapper checks the installed DLL's SHA-256 at native launch time and falls back to FCEUmm joystick mode if it is missing or changed.

Set [VIRTUALGLOVE_NATIVE_STATE](#) to use a test record at a different path. Set [VIRTUALGLOVE_TRACE=1](#) when launching the custom core to log controller writes, latch/counter transitions, returned stream bits, and each candidate

output packet. Traces may contain gameplay timing but no camera imagery.

Exact-ROM validation gate

The exact-ROM software gate below passes for detection, Start, X/Y, Z and hand pose packet publication, safe neutralization, and the tested X/Y cabinet path. A completed live game additionally confirms grab/throw, index fire, and Power Punch. Repeat this gate before enabling the per-ROM emulator choice on another cabinet or after changing the core protocol:

1. Record the ROM digest and retain the ROM outside release packages.
2. Trace controller strobes and configuration writes from power-on through the game's detection decision.
3. Prove the detection signature, packet boundary, bit order, and polling cadence from those traces.
4. Hold every field neutral, then vary X, Y, and Z independently through minimum, center, and maximum values.
5. Transmit open, fist, and index point independently, returning to open between each pose.
6. Confirm repeatable continuous movement plus grab/throw, Robo-Bullet, and fist-plus-forward Power Punch behavior without relying on packet logs alone. The primary cabinet passed this check in a completed game; repeat it after relevant recognition, transport, or core changes.
7. Test stale samples, unavailable calibration, and tracking loss. Stale or uncalibrated input must immediately neutralize; a brief missed observation may hold only X/Y for up to 180 ms, with actions already released, before sustained loss neutralizes coordinates.
8. Build the core with the target system's own toolchain under the separate [lr-nestopia-powerglove/nestopia_powerglove](#) name, verify the camera-to-receiver path, and only then create the per-ROM override.

Keep an explicit FCEUmm per-ROM choice available. If native detection or tracking regresses, remove only the per-ROM override; the shared FCEUmm fallback remains playable.

Compare both modes from EmulationStation

After the custom core has been installed and registered once, RetroPie's launch menu lists both [lr-fceumm](#) and [lr-nestopia-powerglove](#) for this ROM. Choosing [lr-fceumm](#) keeps the entire session in standard joystick mode: it receives the shared Super Glove Ball profile's D-pad, A, B, Start, and Select outputs and does not read the native-state bridge. The native entry attaches device [517](#) and reads continuous coordinates instead.

This switch is automatic after launch. The hook inspects the libretro core that actually started and includes that core in each authenticated renewable profile heartbeat. The Controller enables native output only when both the profile is [super_glove_ball](#) and the core is [lr-nestopia-powerglove](#). FCEUmm, another core, or an unknown core selects joystick output. A core change invalidates the previous output state before the new mode begins, preventing a held direction or native sample from crossing the transition. Dashboard status exposes the reported emulator and the resulting [native](#) or [joystick](#) input mode.

Batocera's game hook normalizes its [nestopia_powerglove](#) core name to the same authenticated [lr-nestopia-powerglove](#) identity. The custom core selects the native peripheral internally, so frontend device timing cannot make the ROM miss its startup detection. This behavior exists only in the separately named core.

Recalbox's bounded process monitor reads the actual [nestopia_powerglove_libretro.so](#) command line and reports that same authenticated identity. Its runtime system-list mount is rebuilt from the current Recalbox template at boot, so an operating-system upgrade does not preserve or overwrite an obsolete copied list.

The launch-menu selection for a ROM is persistent, so a test session should be followed by choosing [lr-nestopia-powerglove](#) again if that is the desired saved default. The command-line selector below performs the same reversible per-ROM choice; neither route changes the system-wide NES emulator.

On the RetroPie host, build/install and then opt in one exact ROM:

```
SH
sudo scripts/install-nestopia-powerglove.sh
sudo python3 scripts/configure-super-glove-ball-core.py \
  --rom "/home/pi/RetroPie/roms/nas/Super Glove Ball (USA).7z" \
  --mode native --apply
```

The selector writes the libretro device value [517](#) and also supplies `--device=1:517` to RetroArch. The explicit launch option ensures this RetroArch build attaches the Power Glove to port 1 before the ROM performs its detection poll. Roll back only that ROM at any time:

```
SH
sudo python3 scripts/configure-super-glove-ball-core.py \
  --rom "/home/pi/RetroPie/roms/nas/Super Glove Ball (USA).7z" \
  --mode fceumm --apply
```